



**GOVERNO DO ESTADO
DE SÃO PAULO**

Faculdade de Tecnologia de Sorocaba
Tecnologia em Análise e Desenvolvimento de Sistemas

RAFAELA MANSANO FERNANDES

TAREFA 6 - ANÁLISE DE PIPELINES

**SOROCABA/SP
2026**

RAFAELA MANSANO FERNANDES

TAREFA 6 - ANÁLISE DE PIPELINES

Esse trabalho foi elaborado como requisito parcial para avaliação na disciplina de DevOps, ministrada pelo Professor Deivison S. Takatu, do curso superior de Análise e Desenvolvimento de Sistemas.

**SOROCABA/SP
2026**

SUMÁRIO

1 INTRODUÇÃO.....	4
2 PIPELINE 1 - Vite: Correção em node_modules.....	5
3 PIPELINE 2 - Vite: otimização de regex no changelog.....	6
4 PIPELINE 3 - React: sincronização de artefatos de runtime.....	7
5 COMPARAÇÃO GERAL ENTRE AS PIPELINES.....	8
6 CONCLUSÃO.....	8

1 INTRODUÇÃO

Este relatório apresenta a análise de três execuções de pipelines de CI/CD no GitHub Actions, extraídas de repositórios de código aberto amplamente utilizados: vitejs/vite e facebook/react. Para cada pipeline foram observadas suas características estruturais, funcionalidades, gatilhos (triggers) de execução e histórico de execução, com base nas telas de resumo disponibilizadas pelo GitHub Actions.

2 PIPELINE 1 - Vite: Correção em node_modules

Repositório: <https://github.com/vitejs/vite/actions/runs/34330479373>

Características:

- Workflow chamado CI, definido no arquivo ci.yml.
- Estruturado em estágios sequenciais: Get changed files → Matrix: test (com 6 jobs) → Build & Test Passed or Skipped.

Funcionalidades:

- Get changed files (10s): identifica quais arquivos foram alterados no PR, otimizando a execução dos testes.
- Lint (2m 0s): checagem de estilo e qualidade de código, testada em node-24 / ubuntu-latest.
- Build & Test em matriz: roda em múltiplas versões do Node (20, 22, 24, 26) e múltiplos sistemas operacionais (Ubuntu, macOS, Windows), garantindo compatibilidade multiplataforma.
- Job final consolida o resultado geral: Build & Test Passed or Skipped (sucesso) ou Build & Test Failed.

Gatilhos:

on: pull_request — a pipeline é disparada automaticamente quando um Pull Request é aberto ou atualizado.

Neste caso: PR #23437, do branch kakiuwang-ui:fix/is-in-node..., sincronizado pelo usuário "bluwy".

Histórico:

Executado há 5 dias em relação à data da captura.

Status: Success, com duração total de 6m 48s.

3 PIPELINE 2 - Vite: otimização de regex no changelog

Repositório: <https://github.com/vitejs/vite/actions/runs/34234994723>

Características:

- Mesma estrutura de pipeline do caso anterior (mesmo ci.yml), o que evidencia a padronização do processo de CI no repositório: todo Pull Request passa pelo mesmo fluxo, independentemente do tipo de mudança.

Funcionalidades:

- Mesmas etapas do Pipeline 1: Get changed files → matriz de Lint / Build & Test → consolidação final.
- A mudança em questão é uma tarefa de manutenção (chore), que evita recompilar uma expressão regular repetidamente no script de changelog — evidenciando que a pipeline roda inclusive para PRs pequenos ou puramente técnicos.

Gatilhos:

on: pull_request — disparado pelo PR #23456, aberto pelo usuário "btea", branch btea:refactor/avoid-re-comp....

Histórico:

Executado na semana anterior à data da captura ("last week").

Status: Success, com duração total de 5m 24s — ligeiramente mais rápida que o Pipeline 1, possivelmente em função do menor impacto dos arquivos alterados.

4 PIPELINE 3 - React: sincronização de artefatos de runtime

Repositório: <https://github.com/react/react/actions/runs/34384325125>

Características:

- Pipeline com propósito distinto das anteriores: não é uma CI de teste/lint, mas sim uma automação de infraestrutura interna, responsável por sincronizar artefatos do build do React com os repositórios internos da Meta (WWW e fbsource).
- Estrutura mais simples e linear, composta por quatro jobs sequenciais.

Funcionalidades:

- `download_artifacts` (38s): baixa os artefatos gerados em outro processo de build.
- `process_artifacts` (16s): processa e prepara esses artefatos.
- `commit_www_artifacts` (6s) e `commit_fbsource_artifacts` (8s): realizam o commit dos artefatos processados em dois destinos internos da Meta, em paralelo.
- A execução gerou 3 artifacts ao final, visíveis no resumo da run.

Gatilhos:

on: `workflow_run` — gatilho diferente dos anteriores: em vez de reagir a um Pull Request, esta pipeline é disparada automaticamente após a conclusão de outro workflow (encadeamento de pipelines).

Disparado pelo usuário "eps1lon", associado ao commit a58f939.

Histórico:

Executado há 5 dias.

Status: Success, com duração total de 1m 17s — significativamente mais rápida que as pipelines do Vite, já que não envolve build ou testes completos, apenas manipulação de artefatos.

5 COMPARAÇÃO GERAL ENTRE AS PIPELINES

Em termos de propósito, as Pipelines 1 e 2 são pipelines de CI voltadas a lint, build e testes multiplataforma, enquanto a Pipeline 3 é uma automação de infraestrutura, focada em sincronizar artefatos de build com sistemas internos.

Quanto ao gatilho, as duas pipelines do Vite são disparadas por `pull_request`, reagindo diretamente a alterações propostas em PRs. Já a pipeline do React é disparada por `workflow_run`.

Sobre a duração, a Pipeline 1 levou 6m 48s, a Pipeline 2 levou 5m 24s, e a Pipeline 3 levou apenas 1m 17s — reflexo direto da diferença de complexidade e do tipo de tarefa executada por cada uma.

6 CONCLUSÃO

Em conjunto, os três exemplos mostram como o GitHub Actions pode ser utilizado tanto para garantir qualidade e compatibilidade de código quanto para automatizar processos de infraestrutura e distribuição de artefatos, adaptando-se a diferentes necessidades dentro do ciclo de vida de desenvolvimento de software.